

From mathematical problems to SciPy solvers

EN.553.285 Introduction to Scientific Programming in Python | Winter 2026

Zan Ahmad · Johns Hopkins University

Edited lecture sample from January 15, 2026 (Lecture 4).

NumPy gives us numerical arrays; SciPy provides algorithms for mathematical tasks. Start by identifying the problem, then choose the solver and inspect its diagnostics. The examples use synthetic data, Python 3, NumPy, and SciPy.

1. Find a root

Suppose $f(x) = x^3 - x - 2$. The function is continuous and has opposite signs at $x = 1$ and $x = 2$, so there is at least one root between them. A sign-changing bracket is a guarantee of existence under continuity, not a guarantee of uniqueness.

```
from scipy import optimize

def f(x):
    return x**3 - x - 2

result = optimize.root_scalar(
    f, bracket=[1.0, 2.0], method="brentq"
)
if not result.converged:
    raise RuntimeError("Root finder did not converge")
print(result.root)
print(abs(f(result.root)))
```

The root is approximately 1.52138. Inspect both the convergence flag and the residual. A small residual is useful evidence, although for a poorly conditioned problem it does not by itself establish a small error in the root.

Why Brent's method is useful

Brent's method maintains a bracket and combines bisection with secant and inverse-quadratic interpolation steps. Faster proposals are accepted when appropriate; bracket-based steps provide a safeguard. The method still depends on the mathematical assumptions and sensible tolerances.

Try it

Evaluate f at both endpoints before calling the solver. What happens if you supply a bracket with no sign change? Explain why a discontinuity can invalidate the existence argument even when endpoint signs differ.

2. Solve an initial-value problem

An ODE specifies how a state changes; an initial condition selects a particular solution. Consider $dx/dt = 0.5x$ with $x(0) = 1$ on $0 \leq t \leq 10$. This example has a known solution, $\exp(0.5t)$, which is useful for checking a numerical solver.

```
import numpy as np
from scipy.integrate import solve_ivp

def growth(t, state):
    return 0.5 * state

solution = solve_ivp(
    growth, (0.0, 10.0), [1.0], method="RK45",
    rtol=1e-7, atol=1e-9, dense_output=True
)
if not solution.success:
    raise RuntimeError(solution.message)
t = np.linspace(0.0, 10.0, 101)
approx = solution.sol(t)[0]
exact = np.exp(0.5*t)
print(np.max(np.abs(approx - exact)))
```

The requested evaluation points do not force the solver's internal steps. RK45 adapts those steps using an embedded pair of approximations to estimate local error. It advances with a fifth-order formula and estimates error using a fourth-order formula.

Choose and check tolerances

Relative and absolute tolerances control the local error estimate in relation to the state's scale. They are not a blanket guarantee about the final global error. Repeat with tighter tolerances, compare the trajectory with the exact solution, and report the change in both error and computational effort.

This smooth example is suitable for RK45. Stiff systems may need other methods. A solver's success flag does not prove that the model, units, initial state, or interpretation are correct.

Try it

Change the growth rate to a negative value and compare with the corresponding decaying exponential. Explain why absolute tolerance becomes important when the solution approaches zero.

3. Interpolate sampled data

Interpolation constructs a function that passes through supplied data points. A cubic spline is piecewise cubic with continuous first and second derivatives. It differs from fitting a model that may intentionally tolerate residual error.

```
from scipy.interpolate import CubicSpline

x = np.array([0.0, 1.0, 2.0, 3.0])
y = np.array([0.0, 1.0, 0.0, 1.0])
spline = CubicSpline(x, y, bc_type="natural",
                    extrapolate=False)
assert np.allclose(spline(x), y)
print(spline(1.5)) # approximately 0.5
```

Natural boundary conditions specify zero second derivative at the two endpoints. They are a modeling choice. Smoothness does not guarantee that the curve stays within the range of the samples or preserves monotonicity. Here extrapolation is disabled explicitly; evaluating outside the sampled interval returns NaN.

Try it

Plot the spline on a fine grid, mark the original samples, and compare it with straight-line interpolation. Change the boundary condition and inspect the endpoints. If the data are noisy, explain why exact interpolation may be less useful than an appropriate fitted model.

A reusable workflow

Define the mathematical problem and assumptions. Choose a numerical representation and solver. Set units and tolerances. Inspect the return status and diagnostics. Compare with a known case, a refined calculation, or an independent formulation before interpreting the result.

Source and revision

Zan Ahmad, Johns Hopkins University, Lecture 4 (January 15, 2026). Edited October 2026: classroom transitions removed; convergence checks, explicit tolerances, an exact ODE comparison, and interpolation limits added. Brent's interpolation step is described more precisely. Course reference: Robert Johansson's Lectures on scientific computing with Python (CC BY 3.0).

Further reading: the official SciPy documentation for `root_scalar`, `solve_ivp`, and `CubicSpline`.

Download the runnable companion at zanahmad.com/files/teaching-samples/python-scipy.py.